Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3676151.3719368

RESEARCH-ARTICLE

Quantifying Data Leakage in Failure Prediction Tasks

DANIEL GRILLMEYER, Julius-Maximilian University of Würzburg, Wurzburg, Bayern, Germany

MARIUS HADRY, Julius-Maximilian University of Würzburg, Wurzburg, Bayern, Germany

VERONIKA LESCH, Baden-Wuerttemberg Cooperative State University, Friedrichstraaye 14, https://www.dhbw.de/startseite.html, Stuttgart

VANESSA BORST, Julius-Maximilian University of Würzburg, Wurzburg, Bayern, Germany

ROBERT LEPPICH, Julius-Maximilian University of Würzburg, Wurzburg, Bayern, Germany

ANDRÉ BAUER, Illinois Institute of Technology, Chicago, IL, United States

View all

Open Access Support provided by:

Julius-Maximilian University of Würzburg

Illinois Institute of Technology

Baden-Wuerttemberg Cooperative State University

PDF Download
3676151.3719368.pdf
17 March 2026
Total Citations: 0
Total Downloads: 393

Published: 05 May 2025

Citation in BibTeX format

ICPE '25: 16th ACM/SPEC International Conference on Performance Engineering
May 5 - 9, 2025
Toronto ON, Canada

Conference Sponsors:
SIGSOFT
SIGMETRICS



Quantifying Data Leakage in Failure Prediction Tasks

Daniel Grillmeyer
University of Würzburg
Würzburg, Germany
Daniel.Grillmeyer@uni-wuerzburg.de

Marius Hadry
University of Würzburg
Würzburg, Germany
Marius.Hadry@uni-wuerzburg.de

Veronika Lesch
Baden-Wuerttemberg Cooperative State University Mosbach
Mosbach, Germany
Veronika.Lesch@dhbw.de

Vanessa Borst
University of Würzburg
Würzburg, Germany
Vanessa.Borst@uni-wuerzburg.de

Robert Leppich
University of Würzburg
Würzburg, Germany
Robert.Leppich@uni-wuerzburg.de

André Bauer
Illinois Institute of Technology
Chicago, USA
Andre.Bauer@iit.edu

Samuel Kounev
University of Würzburg
Würzburg, Germany
Samuel.Kounev@uni-wuerzburg.de

Abstract

With the ever increasing importance of cloud computing and a strong focus on reliable data centers, a high amount of research has been done on failure prediction for hard disk drives. The collection of monitoring data, such as SMART statistics (**S**elf-**M**onitoring, **A**nalysis, and **R**eporting **T**echnology) from operational HDDs, enables operators to obtain predictions about the expected remaining useful life. Numerous methods for HDD failure prediction have been published in recent years, and their evaluation has shown decent results. However, a naive splitting into training and test sets can lead to data leakage and, thus, over-optimistic results that cannot be achieved on the data of scientific interest. In this paper, we propose a novel data leakage measure for quantifying the amount of data leakage in training and test datasets. Further, we define four splitting techniques and evaluate our measure in terms of the performance optimism of classification models with respect to these different splitting strategies. Our results consistently show that splitting techniques prone to data leakage induce an overestimation of predictive performance. Overall, we were able to show the usefulness of the defined data leakage measure, as well as its connection with different splitting techniques and the performance optimism of prediction models.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Computer systems organization** → **Reliability**.

Keywords

Machine Learning, Deep Learning, HDD Failure Prediction, Data Leakage Measure, Data Splitting, Performance Optimism

ACM Reference Format:
Daniel Grillmeyer, Marius Hadry, Veronika Lesch, Vanessa Borst, Robert Leppich, André Bauer, and Samuel Kounev. 2025. Quantifying Data Leakage in Failure Prediction Tasks. In *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE '25), May 5–9, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3676151.3719368

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICPE '25, May 05–09, 2025, Toronto, Canada
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1073-5/2025/05
https://doi.org/10.1145/3676151.3719368

1 Introduction

The growing use of cloud computing in the last decades is projected to continue in the next years, especially with generative artificial intelligence becoming more and more common. According to Gartner, the end-user spending forecast of worldwide public cloud services will increase from 561,104 million U.S. Dollars in 2023 to 824,763 million U.S. Dollars in 2025 [6]. Smooth operation of data centers is therefore a key component for the continued success of these concepts. Besides other data center-related tasks, monitoring and maintenance of Hard Disk Drives (HDDs) is important to ensure reliable operation. Collected monitoring data from HDDs can also be used to perform failure prediction tasks, enabling proactive replacement of failing hard drives.

In the literature, various approaches are proposed to predict failures of HDDs. For example, Pereira et al. [17] made use of one-class classifiers and found that Principal Component Analysis (PCA), k-Means, and bottleneck neural networks (autoencoders) achieved the best results. The majority of papers propose models from the category of Recurrent Neural Networks (RNNs), such as Long Short-Term Memories (LSTMs) [4, 7, 20, 23], to extract information from long input sequences. Additionally, further variations can be identified, like Convolutional Neural Network (CNN) layers added to the front of the LSTM layers, resulting in a CNN-LSTM [8, 12], and LSTMs combined with attention mechanisms [21]. Most of the proposed approaches differ significantly in terms of dataset and preprocessing, especially splitting mechanisms for test and training

dataset. A common issue regarding the reproducibility of the results is that it is often not evident whether care was taken to ensure that no data was leaked between test and training data. We define data leakage, in line with Kaufman et al. [10], as "the introduction of information about the target of a data mining problem that should not be legitimately available to mine from."

The effect and existence of data leakage in publications has already been investigated. For example, Lyu et al. [13] analyzed the impact of different data splitting techniques using the *Backblaze disk stats dataset* [2] and identified a significant effect of the executed splitting strategy on the model performance. Besides, Kaufman et al. [10] analyzed leakage in data mining in general, identified unclean practices in the literature, and presented a formalization of data leakage. Kapoor and Narayanan [9] surveyed 17 prediction task disciplines and found 294 publications affected by data leakage. They presented a taxonomy for the classification of data leakage. Despite the increasing research efforts to identify and formalize data leaks, there are no measurement methods and metrics to quantify the severity of data leakage. As a consequence, there are no studies that measure the influence of data leakage on the performance of prediction models.

With this paper, we aim to fill this gap by posing and discussing the following research questions:

RQ1: *How to mathematically define a measure for quantifying data leakage?*
RQ2: *Which data splitting techniques exist and how severe is the data leakage according to the defined measure?*
RQ3: *How does the consistency of the models' predictive performance change with different splitting techniques and thus different severity of data leaks?*

To answer these research questions, we first define a novel measure for quantifying data leakage and discuss the assumptions made (**RQ1**). The measure addresses the mismatch between the test distribution and the distribution of scientific interest as outlined by Kapoor and Narayanan [9]. Our measure is normalized to the interval $[0, 1]$ and provides a parameter α that allows for distinguishing the behavior of the measure related to the given prediction task. Second, we present four different splitting techniques with varying degrees of data leakage to divide data into training and test datasets (**RQ2**): (i) no split, (ii) random split, (iii) temporal split, and (iv) group-based split. While the *no split* strategy uses the full dataset for training and test, the *random* split strategy assigns measurements randomly to the training and test set. The *temporal* and *group-based* splitting techniques aim at a clean split with no data leakage. Finally, in order to answer **RQ3**, we conduct experiments for all splitting techniques on the Backblaze dataset [2]. We apply a variety of multiclass classification models commonly used in related work, including the ensemble Machine Learning (ML) methods *Histogram-based Gradient Boosting Classifier (HGBC)* and *Random Forest (RF)*, and the Deep Learning (DL) models *Multilayer Perceptron (MLP)* and *LSTM*.

Our evaluation shows that ensemble methods achieve almost perfect results when using the *no split* technique, while DL methods perform slightly worse. This allows the assumption that the models memorize the whole dataset well. When using the *random split* technique, all models perform slightly worse but still fairly well. With *group-based* and *temporal split*, the results are consistently worse. The evaluation on an independent test set using the pre-trained models shows a clear performance deterioration for the *no-split* and *random* splitting techniques. Additionally, the DL methods show a slightly better performance for the *temporal* and *group-based* splitting techniques, while the ensemble method's performance is barely affected by the splitting method choice. We find that the differences between performance on the test set from the respective split and on the independent future set, that is, the Performance Optimism (PO), strongly depend on the splitting method used. Overall, we were able to show the usefulness of the defined data leakage measure as well as its connection with different splitting techniques and PO of prediction models.

The remainder of this paper is structured as follows: First, we discuss related work in Section 2. Afterwards, we fully introduce the term data leakage, propose a novel data leakage measure, and discuss assumptions made in Section 3. In Section 4, we introduce the conducted experiments: The used splitting techniques, the dataset and its preprocessing, the used predictive models and their optimization, the term performance optimism, and the hardware configuration. In Section 5, we present the results of our experiments and discuss important findings, followed by a discussion on threats to validity in Section 6. Finally, we conclude the paper in Section 7.

2 Related Work

The topic of this paper is related to multiple research areas: (i) data leakage, (ii) machine learning models used for HDD failure prediction, and (iii) metrics for quantifying data leakage. However, we were unable to identify any related work on the third aspect. In the following, we discuss the first two points.

2.1 Data Leakage

The effect and existence of data leakage in publications has already been investigated. Lyu et al. [13], for instance, analyzed the impact of different data splitting techniques using the *Backblaze disk stats dataset* [2]. They criticized prior studies for often randomly splitting data into training, validation, and test data. In their study, Lyu et al. [13] compared models trained using random and time-based splitting to separate the available data into training and testing sets. They show that random splitting can introduce data leakage, causing models to perform significantly worse on unseen test data when compared to the results for validation data. Further, they also showed that this effect is limited when employing a time-based splitting approach.

The data leakage problem is not only present in the domain of hard disk failure prediction. Kaufman et al. [10] analyzed leakage in data mining in general. They focused on data mining competitions and studies and investigated the published approaches for data leakage. Kaufman et al. [10] found unclean practices and formalized data leakage and its detection through modeling relations between variables in the dataset. While this approach allows detecting data leakage, it does not provide a meaningful way of quantifying the amount of leakage present. In contrast to Lyu et al. [13], they considered not only data leakage through time but also other aspects like deanonymization and using unavailable external information.

Kapoor and Narayanan [9] surveyed 17 disciplines where machine learning algorithms were employed for prediction tasks and found 294 publications affected by data leakage. They presented a taxonomy for the classification of data leakage and proposed a model info sheet consisting of 21 questions to detect and prevent faulty methodologies. Kapoor and Narayanan [9] claim that a reproducible publication requires a clean train-test separation, reasoning why each feature is legitimate, and a test set that is drawn from the distribution of scientific interest, so that there is no selection or sampling bias.

While Lyu et al. [13] analyzed the impact of data leakage, we additionally focus on quantifying the leakage and analyze if our metric allows us to verify the claims made. Instead of only comparing temporal to random splitting, we also consider group-based splitting, where each HDD serial number defines a group. In addition, we include the case of a missing test dataset, where training and testing are performed on the same data, as a baseline method.

2.2 Models

When it comes to HDD failure prediction using the *Backblaze disk stats dataset* [2], several approaches have already been used. Pereira et al. [17] made use of one-class classifiers. They found that reconstruction methods like Principal Component Analysis (PCA), k-Means and bottleneck neural networks (autoencoders) achieve the best results. Ensemble models like RF also commonly achieved good results [1, 19, 24].

DL-based methods have also been employed for HDD failure prediction. RNNs like LSTMs offer the ability to extract information from long input sequences [4, 7, 20, 23]. Authors have also used variations of the LSTM, like the CNN-LSTM [8, 12], which combines CNNs and LSTMs. Furthermore, Wang et al. [21] combined LSTMs with an attention mechanism.

Opposed to the presented publications in this section, we do not focus on implementing new models or increasing the accuracy of current models for the task. Instead, we show the impact of data leakage using a subset of these models and verify the expressiveness of our proposed metric. We use the work of De Santo et al. [4] and Lu et al. [12] as a basis for our experiments. They provide information on the model inputs and outputs to use regarding sequence lengths and features used.

3 Data Leakage

In this work, we adopt the general definition of *data leakage* provided by Kaufman et al. [10]: "Leakage [in data mining] is the introduction of information about the target of a data mining problem that should not be legitimately available to mine from." [10] Further, we build upon the comprehensive taxonomy proposed by Kapoor and Narayanan [9] that accounts for multiple categories of data leakage. It originates from the findings in their extensive survey on common pitfalls in applying and evaluating ML/DL models. In the following, we summarize the three main cases.

The first case, a *lack of clean separation* of training and test data, is mainly related to pitfalls in the preprocessing pipelines, for instance, that feature selection or the fitting of scalers is performed on the entire dataset instead of only the training data, or that duplicates are not dropped.

The second case refers to *using illegitimate features*, for instance, features acting as a proxy for the outcome variable [10] that would not be available for inference on novel data. For example, in a binary rain prediction task, the information if people carried umbrellas on a specific day could be a proxy for the outcome variable.

The last case is a *distribution mismatch* between the distribution of scientific interest and the test dataset distribution. For instance, models designed to predict a future outcome must not be evaluated on observations prior to those in the training set to prevent temporal leakage. Further, to avoid leakage, training and test samples must be independent, or at least the dependence structure must align with the underlying scientific claim. For instance, if the scientific claim is about the ability of a speech recognition model to transcribe recordings of unseen speakers, the evaluation must not be performed on recordings from speakers included in the training set. Lastly, sampling bias might be introduced if the test sample is not representative of the data of interest. For example, a weather forecasting model intended to be used in North American regions should not be evaluated on data from Australia.

3.1 Data Leakage Measure Definition

In this paper, we focus on the problem of the *distribution mismatch* as outlined by Kapoor and Narayanan [9]. To assess the extent of the present data leakage and to propose an answer to **RQ1**, we define the normalized data leakage measure $\tilde{\mathcal{L}}$ as derived in Equations 1 to 3. The sets $\mathcal{M}_{\text{test}}$ and $\mathcal{M}_{\text{train}}$ denote the test and training datasets. Their union $\mathcal{M}_{\text{test}} \cup \mathcal{M}_{\text{train}}$ is the entire dataset. $\mathcal{D}$ is the set of groups in the dataset, for instance, defined by specific HDDs. $\boldsymbol{X}_{i,t}$ denotes the data sample at timestep t in group i, where $t \in \mathbb{N}$ and $i \in \{1, \ldots, |\mathcal{D}|\}$. $\alpha \in \mathbb{R}_0^+$ is a parameter indicating the leakage impact of training samples preceding test samples, as explained below (Case 2).

$$\mathcal{L}(\mathcal{M}_{\text{test}}, \mathcal{M}_{\text{train}}) := \sum_{i=1}^{|\mathcal{D}|} \sum_{\substack{\boldsymbol{X}_{i,t_1} \\ \in \mathcal{M}_{\text{test}}}} \sum_{\substack{\boldsymbol{X}_{i,t_2} \\ \in \mathcal{M}_{\text{train}}}} l(\boldsymbol{X}_{i,t_1}, \boldsymbol{X}_{i,t_2}) \tag{1}$$

$$l(\boldsymbol{X}_{i,t_1}, \boldsymbol{X}_{i,t_2}) := \begin{cases} e^{-(t_2 - t_1)}, & \text{if } t_2 \geq t_1 \\ \alpha e^{-(t_1 - t_2)}, & \text{if } t_2 < t_1 \end{cases} \tag{2}$$

$$\tilde{\mathcal{L}} = \frac{\mathcal{L}(\mathcal{M}_{\text{test}}, \mathcal{M}_{\text{train}})}{\mathcal{L}(\mathcal{M}_{\text{test}}, \mathcal{M}_{\text{test}})}, \; \tilde{\mathcal{L}} \in [0, 1] \tag{3}$$

In Equation 1, for each group i, the leakage contribution of each pair of test data points $\boldsymbol{X}_{i,t_1} \in \mathcal{M}_{\text{test}}$ and train data points $\boldsymbol{X}_{i,t_2} \in \mathcal{M}_{\text{train}}$ is summed up. The pair-wise leakage contribution $l(\boldsymbol{X}_{i,t_1}, \boldsymbol{X}_{i,t_2})$ is defined in Equation 2. It is designed such that a small temporal proximity between train and test data points has a large impact on the overall data leakage, while the impact decreases exponentially for increasing time differences. An example for the calculation of the pair-wise leakage contribution is illustrated in Figure 1. Equation 2 distinguishes two cases:

Case 1 ($t_2 \geq t_1$) The test sample *precedes* the training sample. This introduces temporal leakage as described in Section 3.

Case 2 ($t_2 < t_1$) The impact of the second case where the test sample *proceeds* the training sample is controllable by parameter α.

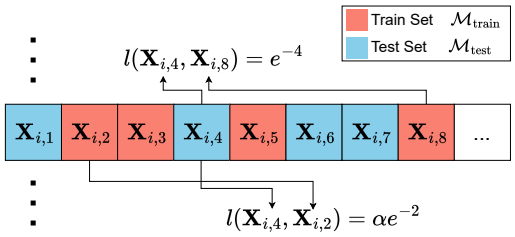


Figure 1: Example for the calculation of the leakage contribution for two pairs of test and train observations.

The parameter α should be chosen in alignment with the task for which a claim is being made. If the scientific claim refers to the predictive performance on novel groups not seen in the training data, it should be set to $\alpha > 0$ such that training data of the same group, even if lying in the past, has a contribution to the leakage value $\mathcal{L}$. For $\alpha = 1$, preceding training samples (Case 2) and proceeding training samples (Case 1) of the same group contribute equally to $\mathcal{L}$.

If the scientific claim refers to the predictive performance on data of groups also used to mine from, it should be set to $\alpha = 0$. Thus, preceding training data of the same group is legitimate and accordingly does not contribute to $\mathcal{L}$.

Lastly, to obtain the final normalized leakage value $\tilde{\mathcal{L}} \in [0, 1]$ in Equation 3, we normalize the sum of all pair-wise leakage contributions by factor $\mathcal{L}(\mathcal{M}_{\text{test}}, \mathcal{M}_{\text{test}})$. The normalization factor corresponds to the case that all available data is used both for training and testing, yielding the maximum sum of leakage contributions. In the experiments in Section 4.5, we apply the data leakage measure $\tilde{\mathcal{L}}$ on the *Backblaze disk stats dataset* [2].

3.2 Assumptions

Although our proposed data leakage measure is not limited to a specific task or dataset, it depends on multiple assumptions, which are addressed below.

(1) As the measure is based on the difference of timestamps, it requires that the correct timestamps for all observations in the dataset are available. If timestamps are absent, the data must instead consist of ordered sequences of equidistant observations, allowing the ascending enumeration to serve as a proxy for timestamps.
(2) According to Equation 1, only pairs of train and test samples within the same group contribute to the leakage value $\mathcal{L}$. Thus, the underlying assumption is that pairs of samples of different groups are independent and are thus not susceptible to leakage.
(3) The leakage contributions are independent of the feature values, but solely depend on the time difference.
(4) The measure can only represent leakage introduced by the last category in the taxonomy by Kapoor and Narayanan [9], that is, a *mismatch between the distributions*. Leakage caused by the use of illegitimate features, feature selection on training and test data, duplicates in the dataset, or the incorrect application of preprocessing techniques, for instance, fitting scalers to the entire data instead of just the training data, is out of scope. Therefore, the underlying assumption is that these pitfalls are avoided when preprocessing the data before applying the measure.
(5) The leakage measure relies on group labels assigned to the data samples. It is assumed that there is at least one group and that temporal dependencies between data samples within the same group exist.

4 Experiments

In this section, we describe the design of our experiments towards answering **RQ2** and **RQ3** in detail. Section 4.1 presents the different data split strategies. Section 4.2 introduces the dataset used for the experiments and defines the classification task. Section 4.3 explains the conducted preprocessing steps. Section 4.4 addresses the adopted ML/DL methods, hyperparameter optimization, model training, and evaluation process. Section 4.5 explains how the PO measurements are conducted. Section 4.6 provides details about the hardware and software configuration. The overall experiment workflow is illustrated in Figure 2.

4.1 Split Strategies

In accordance with **RQ2**, we present four different strategies to split a dataset into training and test sets. In each of the split strategies except *no split*, we employ a train/test split ratio of 70/30, as this is a common choice in prior studies [17, 19, 23]. The examination of the impact of different splitting ratios as a further degree of freedom is beyond the scope of this paper. Rather, we aim to investigate the impact of different split strategies on the resulting test performance and their respective extent of data leakage.

The strategies are illustrated in Figure 3, where each horizontal bar illustrates the sequence of all available observations $\boldsymbol{O_i}$ for a specific HDD i. The colors indicate the portions assigned to the training set (red) and the test set (blue). The first strategy, *no split*, is intended to serve as a baseline where the whole dataset is used both for training and evaluating the model's predictive performance. The second strategy, *random split*, separates the data randomly into train and test sets, regardless of the temporal or group information. The third strategy, *temporal split*, is designed to prevent temporal leakage. All available data before and after t_{split} is used as training and test data, respectively. The last strategy, *group-based split*, is based on groups that depend on the specific domain, for instance, defined by different HDDs present in the dataset. Each group and its associated data are assigned to the training or test data, but not both. Thus, this method is designed to avoid leakage due to non-independence between training and test samples.

Please note that the training data is further split into one portion used for fitting the trainable parameters with the optimizer, and one validation set used to track the model's predictive performance and apply early stopping when the validation loss stops decreasing. For this split, the same method is used as for the prior split, that is, we further subdivide the training part randomly (b), split it into two temporal components based on another $t_{\text{split}}^{(v)}$ (c), or sub-select some hard disks as the validation part (d). For the actual split strategies, we employ a train/validation/test ratio of 60/10/30. For the sake of

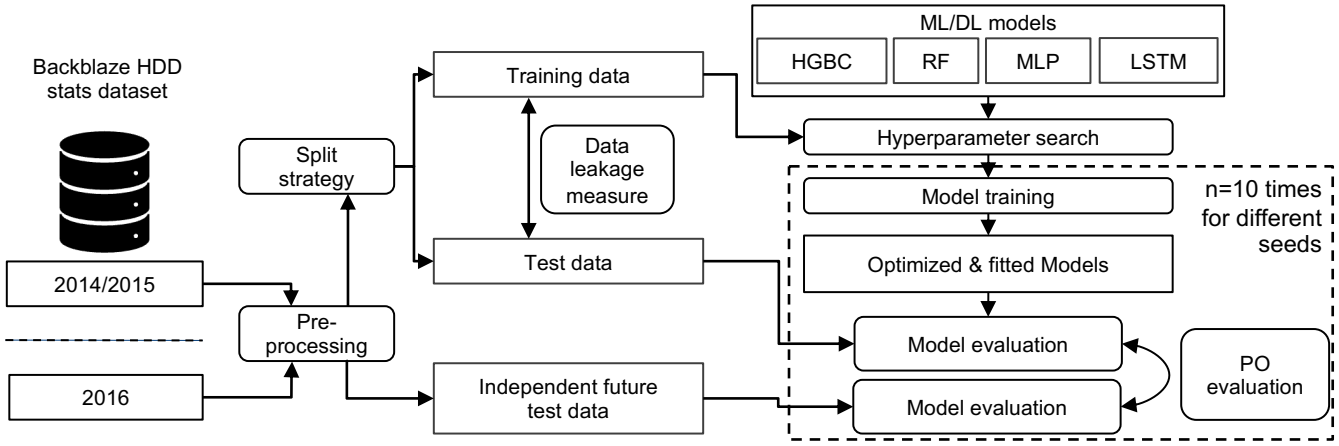


Figure 2: Overview of the experiment workflow. The Backblaze [2] HDD datasets from 2014/15 and 2016 undergo preprocessing independently. The 2014/15 dataset is then split into training and test data using four different split strategies, on which our data leakage measure is applied. For each split strategy and each ML/DL model, a hyperparameter search is conducted, followed by $n = 10$ training runs with different random seeds and subsequent model evaluation. Additionally, the trained models are evaluated on the independently processed 2016 dataset. Based on these evaluations, performance optimism is analyzed.

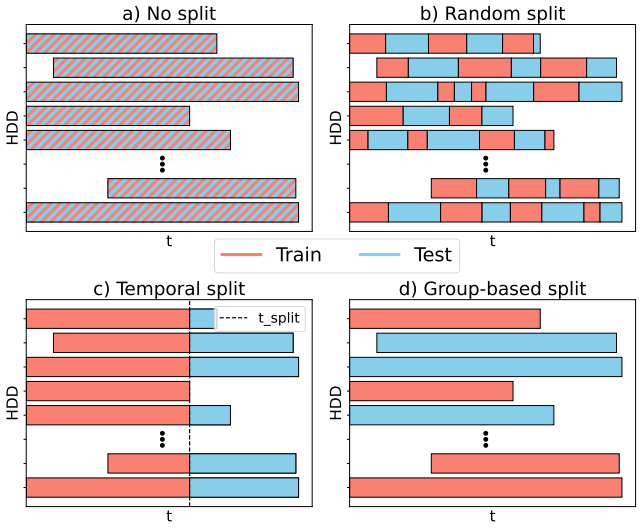


Figure 3: Illustration of the data split strategies.

simplicity, we refer to the training and validation sets together as the *training set* in the remainder of this paper, since both serve the purpose of optimizing the models.

4.2 Dataset and Problem Definition

This paper utilizes the widely used *Backblaze disk stats dataset* [2] that has been publicly available with regular releases since 2013. For each available HDD, uniquely identifiable by its serial number, there are 124 columns of Self-Monitoring, Analysis and Reporting Technology (SMART) statistics, 62 columns each for raw and normalized values, and a failure column with a binary health status indicating whether it is still operational on the next day or not.

The data splitting strategies used in the experiments are applied to the HDD data from 2014 and 2015, even though more recent releases are available. We make this choice to enable comparability with related work using this time period [1, 4, 17, 19]. Note that more up-to-date results are provided in a publicly available repository[1]. Similarly, we limit ourselves to the commonly used HDD model ST4000DM000 [4, 7, 8, 17, 19]. It shows comparably many failures (823) during the observation period, a high annualized failure rate (2.96%), and the highest number of drive days (10,154,343) in the dataset. Theoretically, annualized failure rates can be larger than 100%. Values larger or equal to 100% would imply that a drive lasts one year or less on average [2].

The set of all available HDDs $\mathcal{D}$ can be disjunctly divided into the sets $\mathcal{D}_{\text{fail}}$ and $\mathcal{D}_{\text{healthy}}$ of HDDs failing in the observation period and those that do not, respectively. The former are underrepresented in the data as reflected by the ratio $|\mathcal{D}_{\text{fail}}|/|\mathcal{D}| = 2.75\%$. For each HDD d_i, there is a sequence of observations $\boldsymbol{O}_i = \boldsymbol{o}_{i,1}, \ldots, \boldsymbol{o}_{i,\tau_i}$, where τ_i denotes the last recorded day. In the case of $d_i \in \mathcal{D}_{\text{fail}}$, τ_i is the last operational day.

We define the Remaining Useful Life (RUL) as the number of remaining operational days from time t for HDD d_i, including the current day, as depicted in Equation 4.

$$\text{RUL}(\boldsymbol{o}_{i,t}) = \begin{cases} \infty & \text{if } d_i \in \mathcal{D}_{\text{healthy}} \\ \tau_i - t + 1 & \text{if } d_i \in \mathcal{D}_{\text{fail}} \end{cases} \tag{4}$$

To assign the observations a label for multiclass classification, four health states based on the RUL are defined in Equation 5. We define the *critical* class by a RUL of seven days or less. This aligns with a Backblaze article by Wilson [22], which states that seven days of "data rebuild time" is sufficient for a failed 4 TB HDD. Four weeks or less RUL should yield a warning. A higher RUL value indicates a fair or good health state.

[1] https://github.com/DescartesResearch/Data-Leakage-HDD-Failure-Prediction

Table 1: Selected SMART features for HDD failure prediction.

ID	Attribute Description	ID	Attribute Description
1	Spin-Up Time	189	UltraDMA CRC Error Count
3	Reallocated Sectors Count	192	Read Error Rate
4	Seek Error Rate	193	Start/Stop Count
5	Power-On Hours	194	Spin Retry Count
7	Power Cycle Count	195	Reported Uncorrectable Errors
9	High Fly Writes	197	Load Cycle Count
10	Power-off Retract Count	198	Hardware ECC Recovered
12	Temperature	199	Current Pending Sector Count
187	Uncorrectable Sector Count		

$$c(\boldsymbol{o}_{i,t}) = \begin{cases} \text{"good"} & \text{if RUL}(\boldsymbol{o}_{i,t}) = \infty \\ \text{"fair"} & \text{if RUL}(\boldsymbol{o}_{i,t}) > 28 \\ \text{"warning"} & \text{if RUL}(\boldsymbol{o}_{i,t}) > 7 \text{ and RUL}(\boldsymbol{o}_{i,t}) \leq 28 \\ \text{"critical"} & \text{if RUL}(\boldsymbol{o}_{i,t}) \leq 7 \end{cases} \tag{5}$$

We define the multiclass HDD failure prediction problem as follows. Given observations $\boldsymbol{X}_{i,t}$ associated with HDD d_i and time t, the goal is to infer the corresponding health state $c(\boldsymbol{o}_{i,t})$ from $\boldsymbol{X}_{i,t}$ which is a sequence of T equidistant observations as depicted in Equation 6. We set $T = 8$ throughout the experiments, which is similar to the study by Lyu et al. [13], who use a one-week history.

$$\boldsymbol{X}_{i,t} = \boldsymbol{o}_{i,t-T+1}, \ldots, \boldsymbol{o}_{i,t-1}, \boldsymbol{o}_{i,t} \tag{6}$$

In addition to the classes defined in Equation 5 that are used for model training, similar to prior work [4], we define a coarse-grained partitioning c_{cg} into two classes for binary classification evaluation, as formalized in Equation 7. The *failure* class indicates an impending failure within one week, while the *operational* class indicates a RUL of more than one week.

$$c_{\text{cg}}(\boldsymbol{o}_{i,t}) = \begin{cases} \text{"operational"} & \text{if RUL}(\boldsymbol{o}_{i,t}) > 7 \\ \text{"failure"} & \text{if RUL}(\boldsymbol{o}_{i,t}) \leq 7 \end{cases} \tag{7}$$

We intend to use the fine-grained labels c for model fitting to enable the ML and DL methods to obtain more detailed supervision and feedback in the training process and facilitate learning more subtle differences between different health states. The coarse-grained labels c_{cg}, on the other hand, allow a better understandable and interpretable evaluation of the predictive performance, as established binary classification metrics can be applied. The partitioning is based on practical considerations in the Backblaze article by Wilson [22].

4.3 Data Preprocessing

The preprocessing pipeline used throughout the experiments consists of multiple steps, as explained in the following:

In the first preprocessing step, we identify and, if necessary, remove possible duplicate observations in the dataset to ensure that each sequence $\boldsymbol{X}_{i,t}$ is unique. However, for the aforementioned HDD model and time periods, we did not find duplicates in the dataset.

Due to the large dataset size, the training of DL models is computationally expensive and time-consuming. To make the experiments more feasible, we reduce the portion of the majority class in the second preprocessing step by random uniform sampling of 2.5% of the *good* HDDs. The dataset's class distribution before and after sampling is shown in Figure 4. Without sampling, it reveals a clear imbalance. The class *good* makes up 97.6% of the data, while the most crucial class *critical* accounts for only 0.0557% of the data. After sampling, the dataset size is decreased by 95.28%, and the class imbalance is reduced. Despite the possibility of thereby introducing a sampling bias, we assume the *good* class, still being the largest class, remains sufficiently represented in the dataset.

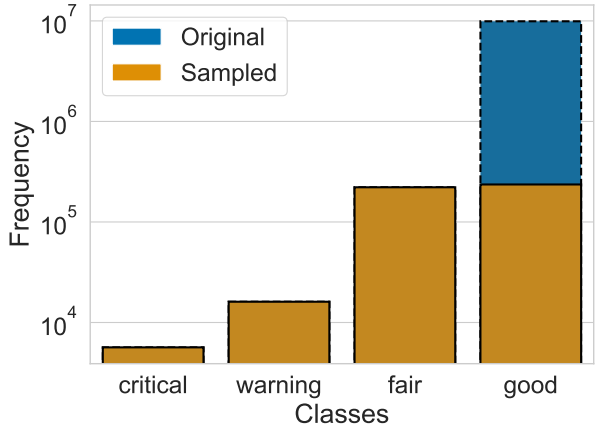


Figure 4: Class distribution of the Backblaze HDD stats dataset (2014/15) with logarithmic frequency scale.

Third, the SMART features are selected based on prior work [4, 12] on HDD failure prediction and SMART feature selection. In addition, features with a missing data rate of more than 20% are discarded, as we aim to rely on reliably recorded features only. The adopted features are listed in Table 1. This includes both the *raw* and *normalized* versions of these features as provided by Backblaze [2]. We assume that the well-known and established SMART measurements are legitimate features since these can be continuously measured. Thus, their availability for inference can be presumed. Further, to the best of our knowledge, none of the features is a proxy for the outcome variable, as it would possibly be the case for the SMART 231 feature *(Life Left)* for Solid State Drives (SSDs).

In the fourth preprocessing step, missing values are handled. Observations $\boldsymbol{X}_{i,t}$ with at least one missing SMART feature value are dropped. We perform this step after the feature selection to avoid dropping observations with missing values of features that are discarded anyway.

Fifth, zero-variance features are discarded since they possess no predictive power. Sixth, the data is split into training and test data, depending on which of the split strategies proposed in Section 4.1 is selected.

Lastly, the data is scaled. Each of the listed features in Table 1 is included with *raw* and *normalized* values as provided by Backblaze [2]. We scale the *raw* SMART features with the QuantileTransformer by scikit-learn [16] because it is robust against extreme outliers

present in some *raw* features. Additionally, we standardize the *normalized* SMART features with their mean and standard deviation. To prevent data leakage in the scaling process, we exclusively fit the scalers on the training data.

4.4 Model Validation

Predictive Models We choose a variety of multiclass classification models that are commonly used in related work, including ML ensemble and DL models. The selection of methods includes *HGBC*, *RF*, *MLP*, and *LSTM*. For the *HGBC* and *RF*, we choose the implementation in the *scikit-learn* [16] Python package. The *MLP* and *LSTM* are implemented in PyTorch [15]. For the training procedure, the PyTorch Lightning [5] framework is used with the *AdamW* [11] optimizer, *weighted Cross Entropy (CE) loss* and early stopping. The weights of the CE loss are proportional to the inverse frequency of the labels in the training data. The batch size is 32, the early stopping patience is 15 and the minimum delta is 0.01, while the best learning rate is determined via hyperparameter optimization.

Hyperparameter Optimization To ensure a fair comparison of the different ML and DL methods, the Bayesian optimizer Hyperopt [3] is used to explore the hyperparameter space and find the configuration with the best validation loss for each model and data split strategy. The number of required trials is estimated by following the guidance by Owen [18]. The configurations found to deliver the best validation loss are used in the following experiments. More details can be found in the appendix in Section B.

Training and Evaluation In the next step of the experiments, the training is repeated $n = 10$ times for each model and splitting strategy, varying the random seed for each of the runs. This procedure allows reliable statements about the performance of the multiclass prediction methods on the test set by eliminating statistical fluctuations due to randomness in the model weight initialization or the data splitting, for example. This enables an accurate evaluation of the predictive performance and a fair comparison. The test results from multiple runs are aggregated by computing the mean and sample standard deviation. The latter is preferred over the usual population standard deviation, as the results of the $n = 10$ runs represent a random sample of all possible random initializations.

$$
\begin{aligned}
\text{precision} &:= \frac{\text{TP}}{\text{TP} + \text{FP}} \\
\text{recall} &:= \frac{\text{TP}}{\text{TP} + \text{FN}} \\
\text{specificity} &:= \frac{\text{TN}}{\text{TN} + \text{FP}} \\
\text{F1} &:= 2\frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \\
\text{G}_{\text{mean}} &:= \sqrt{\text{recall} \times \text{specificity}} \\
\text{MCC} &:= \frac{\text{TP} \times \text{TN} - \text{FP} \times \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}}
\end{aligned}
\tag{8}
$$

We quantify the predictive performance by standard classification metrics. These include F1 score, G_{mean} and Matthews Correlation Coefficient (MCC). The definition of these metrics is provided in Equation 8. TP and FP denote true and false positives, TN and FN true and false negatives, respectively. Positives correspond to the *failure* label, while negatives correspond to *operational* samples (see Equation 7).

4.5 Performance Optimism

As discussed in Section 3, the test set on which ML and DL models are evaluated to prove a scientific claim must be representative of the data the claim is stated for. Otherwise, a model's abilities on the test set might strongly deviate from the performance in a real-world scenario, possibly leading to unrealistic and over-optimistic results and expectations, denoted as Performance Optimism (PO). To find an answer to **RQ3**, we measure and compare the PO of different splitting strategies and ML and DL models. Therefore, in addition to the test sets obtained by the different splitting strategies, we define a separate and independent test set that is assumed to be representative of the data of scientific interest.

The independent test set consists of the *Backblaze disk stats dataset* [2] for the year 2016. Thus, it is guaranteed that all training and test data used in the previous experiment steps is older than this dataset.

The PO is defined as the difference between the models' predictive performance on the test set that is obtained via the respective splitting strategy applied to the HDD stats data from 2014/2015, and the performance on the independent test set from 2016.

Further, to validate our proposed data leakage measure $\tilde{\mathcal{L}}$, we apply Equation 3 to the training and test sets obtained by the different splitting strategies. We examine whether our proposed measure's values align with the measured PO.

4.6 Hardware and Software Configuration

The experiments are conducted on a server featuring two AMD EPYC 7763 64-core processors @2.45 GHz and 256GB of RAM. Additionally, the server houses two NVIDIA A100 GPUs with 80GB VRAM. Each GPU is split into three virtual devices with different VRAM capacities. For the DL experiments, a 40GB instance of *GPU 2* is used. The server runs Ubuntu 22.04.2.

The project code is written in Python and publicly available to foster reproducibility[2]. We use Python 3.11 to run the experiments. Details about the used Python packages and their versions are listed in the appendix in Section A.

5 Evaluation

In Section 5.1, we apply our proposed measure for quantifying data leakage (**RQ1**) on the splitting strategies presented in Section 4.1 to evaluate their extent of data leakage and answer **RQ2**. In Section 5.2, we assess the models' predictive performance based on different splitting strategies and additionally on an independent future test dataset. In Section 5.3, we evaluate the consistency of the models' predictive performance, that is, examine the existence of PO concerning different splitting strategies and classification metrics (**RQ3**). Further, we relate these results to the extent of data leakage indicated by our proposed measure.

[2] https://github.com/DescartesResearch/Data-Leakage-HDD-Failure-Prediction

5.1 Data Leakage Measure

To evaluate our proposed data leakage measure, we employ the four presented data splitting techniques on the *Backblaze disk stats dataset* [2] from 2015/2016. Then, we apply our measure (Equation 3) on the different splits to answer **RQ2**. As described at the end of Section 4.1, the training set also includes the validation set for the computation. Table 2 shows the normalized data leakage value $\tilde{\mathcal{L}}$ for the different data splitting techniques and two different α values 0 and 1. As described in Section 3.1, these two values correspond to different prediction tasks. For $\alpha = 0$, training samples that are lying temporally before a test sample of the same group are not considered leaks. Therefore, it corresponds to application scenarios where predictions in the future are made for groups that are also used to train the model (use case 1). For $\alpha > 0$, on the contrary, these aforementioned training samples are considered leaks. This corresponds to making predictions for novel groups not seen during training (use case 2). We choose $\alpha = 1$ so that these samples' leakage contribution is weighted equally to temporally leaked training samples. Our goal is to validate the measure conceptually. The two selected α values demonstrate its applicability and behavior, while an exhaustive parameter study is beyond the scope of this paper, as further discussed in Section 6.

In the baseline strategy *no split* (Figure 3 - a), all available data is used both as training and test sets. This implies that for each test sample, all existing future observations must be in the training set, thus leading to maximum temporal leakage. Further, the maximum dependency between train and test samples exists since each test sample is also present in the training set. These properties are reflected by the maximum normalized leakage value of 1.0, since the metric is normalized on this extreme case (first row in Table 2).

In the *random split* strategy (Figure 3 - b), all available data is split regardless of temporal or group information. This might lead to temporal leakage, since for each test sample, at a train/test ratio of 70/30, 70% of all existing future observations are statistically expected to be in the training set. Further, non-independence between train and test samples might be introduced due to close temporal proximity of samples within groups. This leads to a dependence structure that does not exist between training data and novel future data. The normalized leakage value reflects these leakage issues with a value of 0.34 for $\alpha = 0$ and 0.59 for $\alpha = 1$ (second row in Table 2). The value is lower for $\alpha = 0$ because it is based on the assumption that data from a group is allowed to be split between training and test data (use case 1). For $\alpha = 1$, on the other hand, the value is higher because this corresponds to the stricter scenario (use case 2), where also training samples that precede test samples, if in the same group, contribute to the leakage value.

Table 2: Data leakage measure results.

Split	Data Leakage $\tilde{\mathcal{L}}$	
	$\alpha = 0$	$\alpha = 1$
No split	1.0	1.0
Random	0.34	0.59
Temporal	0.0	2.0×10^{-3}
Group-based	0.0	0.0

The *temporal split* technique (Figure 3 - c) performs a clean temporal separation at a certain point in time t_{split}. This prevents temporal leakage as it guarantees that no future training examples exist for each test sample. Correspondingly, in the case of $\alpha = 0$, the leakage measure value is 0.0. However, the *temporal split* does not respect the dependence structure of the stricter task of predicting failures for novel unknown groups. This is because the test data contains data of the same groups as the training data. Correspondingly, the measure yields a value > 0 (third row in Table 2).

Lastly, the *group-based split* strategy (Figure 3 - d) guarantees that data belonging to the same group is not split between train and test set. Under the assumption that no dependencies between the different groups exist (see Section 3.2), for instance, that each HDD wears out independently over time, this ensures independence between train and test samples. Consequently, no future training sample in the same group can exist for each test sample, ruling out temporal leakage. This is reflected by a zero leakage value, independent of the choice of α (last row in Table 2).

5.2 Model Validation

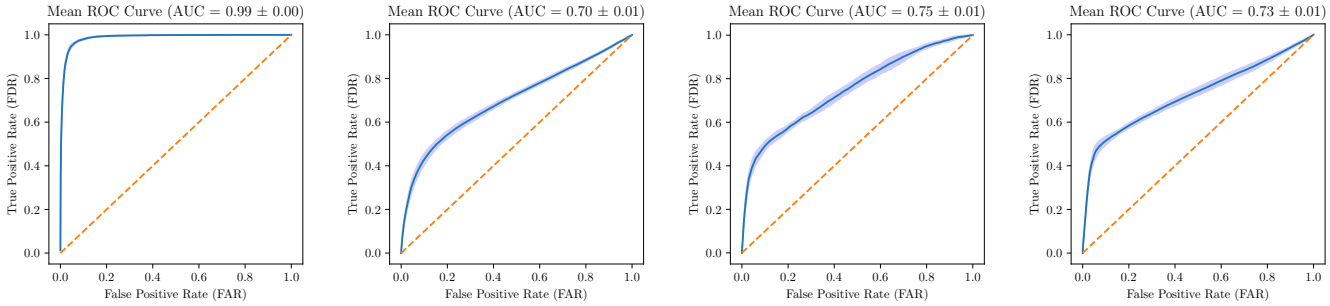
The mean value and sample standard deviation from $n = 10$ runs with different random seeds were calculated for reliable results. Table 3 shows the evaluation results for the different models, splitting techniques, and classification metrics. The values in the first four rows refer to the test performance on the test sets obtained by the splitting techniques (Section 4.1) applied to the 2014/2015 HDD stats data. For the *no split* baseline, the ensemble methods, especially RF, reach superior results, indicating that they perfectly memorized the training dataset. The DL-based methods also show near-perfect scores; however, with deductions for F1 and MCC for the MLP model. For the *random split*, the models still show promising results, especially RF with an G_{mean} of 0.91 ± 0.01, while LSTM and MLP are slightly worse with 0.86 ± 0.01 and 0.81 ± 0.05, respectively. **For the remaining splitting techniques *temporal* and *group-based*, a clear drop in test performance is evident across all models and metrics.** Particularly noteworthy is the deterioration of the MCC for RF and the *temporal split*, which is reduced by 0.75 compared to the *random split*. In comparison, the differences between *temporal* and *group-based* split are rather negligible.

Next, the trained models optimized for the different split strategies are tested on the independently processed future test data with HDD stats data from 2016. Let $\mathcal{D}_{2016}$ be the corresponding set of HDDs of the same HDD model ST4000DM000 as before. A comparison with the set of HDDs $\mathcal{D}_{2014/2015}$ in the 2014/2015 dataset shows that $\frac{|\mathcal{D}_{2014/2015} \cap \mathcal{D}_{2016}|}{|\mathcal{D}_{2016}|} = 81.5\%$ of the HDDs were already present in the previous dataset.

The results on the 2016 data are shown in the latter four rows in Table 3. For the DL-based models MLP and LSTM, the performance of the models trained with the *no split* and *random split* strategies show an overall worse performance than those from the other two strategies. This meets the expectation because the models trained with the *temporal* or *group-based* strategies might be more likely optimized to generalize to unknown future periods or HDDs due

Table 3: Performance of models trained and tested with different split strategies applied to the 2014/15 dataset and performance on the held-out future test data from 2016.

	HGBC			RF			MLP			LSTM		
Split	F1	G_{mean}	MCC	F1	G_{mean}	MCC	F1	G_{mean}	MCC	F1	G_{mean}	MCC
2014/2015 (Train & Test)												
No split	0.94 ± 0.07	0.99 ± 0.00	0.94 ± 0.06	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.71 ± 0.03	0.96 ± 0.01	0.73 ± 0.03	0.93 ± 0.03	0.99 ± 0.00	0.93 ± 0.02
Random	0.59 ± 0.05	0.75 ± 0.01	0.59 ± 0.05	0.87 ± 0.01	0.91 ± 0.01	0.87 ± 0.01	0.39 ± 0.09	0.81 ± 0.05	0.42 ± 0.08	0.53 ± 0.02	0.86 ± 0.01	0.55 ± 0.02
Temporal	0.17 ± 0.01	0.42 ± 0.03	0.16 ± 0.01	0.12 ± 0.01	0.34 ± 0.02	0.12 ± 0.01	0.09 ± 0.02	0.64 ± 0.03	0.12 ± 0.02	0.10 ± 0.03	0.63 ± 0.05	0.13 ± 0.02
Group-based	0.14 ± 0.02	0.48 ± 0.02	0.14 ± 0.01	0.14 ± 0.02	0.43 ± 0.03	0.14 ± 0.02	0.13 ± 0.02	0.67 ± 0.02	0.17 ± 0.01	0.14 ± 0.01	0.67 ± 0.02	0.17 ± 0.01
2016 (Test only)												
No split	0.14 ± 0.04	0.54 ± 0.04	0.15 ± 0.04	0.15 ± 0.02	0.54 ± 0.02	0.15 ± 0.01	0.10 ± 0.01	0.58 ± 0.02	0.12 ± 0.01	0.12 ± 0.01	0.56 ± 0.02	0.13 ± 0.01
Random	0.16 ± 0.02	0.55 ± 0.02	0.16 ± 0.01	0.15 ± 0.02	0.53 ± 0.02	0.16 ± 0.02	0.09 ± 0.01	0.62 ± 0.02	0.12 ± 0.01	0.11 ± 0.01	0.60 ± 0.03	0.13 ± 0.01
Temporal	0.14 ± 0.02	0.50 ± 0.06	0.14 ± 0.03	0.14 ± 0.02	0.47 ± 0.01	0.14 ± 0.01	0.12 ± 0.04	0.67 ± 0.02	0.15 ± 0.03	0.12 ± 0.05	0.66 ± 0.02	0.15 ± 0.05
Group-based	0.11 ± 0.02	0.51 ± 0.06	0.11 ± 0.02	0.14 ± 0.02	0.55 ± 0.03	0.15 ± 0.02	0.13 ± 0.04	0.69 ± 0.01	0.17 ± 0.04	0.17 ± 0.02	0.70 ± 0.01	0.21 ± 0.02



(a) Random split 2014/15 (Train & Test) **(b) Random split 2016 (Test only)** **(c) Temporal split 2014/15 (Train & Test)** **(d) Temporal split 2016 (Test only)**

Figure 5: ROC curves of LSTM trained and tested with two splitting strategies applied to the 2014/15 dataset and respective performance on the held-out future test data from 2016. The shaded areas indicate the sample standard deviation.

to the split choice and the corresponding nature of the validation data.

In contrast, the ensemble methods HGBC and RF have the tendency to achieve better results on the 2016 dataset when trained on the whole 2014/2015 dataset, that is, with the *no split* strategy. Consequently, they show more robustness against overfitting than the DL methods and seem to profit from more training examples. Overall, the ensemble methods' performance on the independent test data is barely affected by the choice of splitting method in the training procedure. **Nevertheless, depending on the chosen split method, the predictive power might be drastically overestimated**, as further discussed below.

Most importantly, the consistency of the performance between the independent evaluation on the 2016 dataset and the 2014/15 dataset strongly depends on the adopted splitting strategy. This can also be seen in comparing the ROC curves of the models. To exemplify, we show the ROC curves of the LSTM in Figure 5. If the *random split* is used, an almost perfect performance with AUC = 0.99 is obtained on the test dataset. However, this outstanding result cannot be confirmed on the independent future dataset, where we observe a significant drop to AUC = 0.70 ± 0.01. On the other hand, if the temporal split is used, the performance on the test (AUC = 0.75 ± 0.01) is much more consistent with the result on the independent future dataset (AUC = 0.73 ± 0.01). The use of the *random split*, therefore, leads to a drastic overestimation, that is, Performance Optimism (PO), of the model's capabilities. This will be further investigated in Section 5.3.

5.3 Performance Optimism

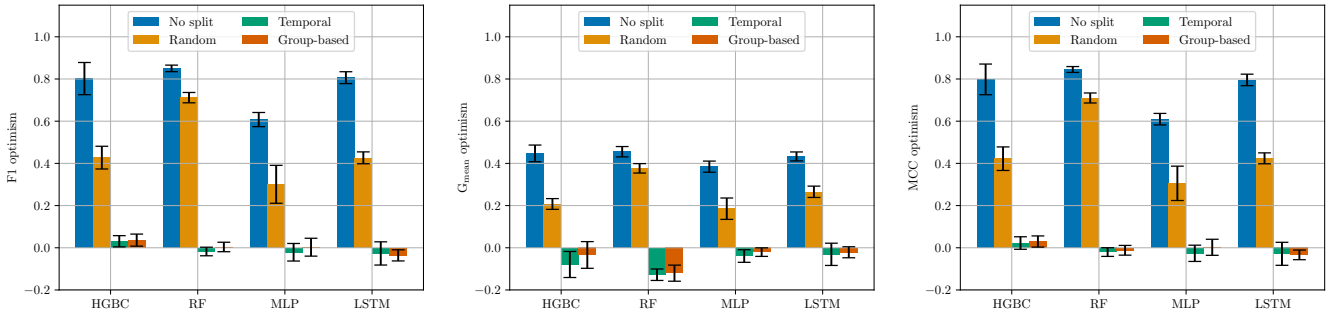
The goal of this section is to find an answer to **RQ3** by comparing the predictive performance of multiple ML/DL models for different splitting strategies and on an independent held-out dataset.

Table 4 provides the PO of the different splitting strategies across multiple models. Based on prior work [13], the PO for a certain metric is defined as the difference between the metric values from the evaluation on the test set from the splitting strategy and the independently processed future test data, respectively. Therefore, positive PO values (in red) indicate the overestimation of a model's predictive capabilities, while zero or negative values (in green) indicate a realistic or under-estimation.

The first two rows in Table 4 show clearly that the results obtained from *no split* and *random split* cannot hold on the independent test set. For the baseline technique *no split*, the F1/MCC and G_{mean} optimism is up to 0.85 and 0.46, respectively. **These**

Table 4: Performance Optimism (PO) of different split strategies. Red values indicate an overestimation of the predictive power, green values indicate a realistic or underestimation.

	HGBC			RF			MLP			LSTM		
Split	F1	G_{mean}	MCC	F1	G_{mean}	MCC	F1	G_{mean}	MCC	F1	G_{mean}	MCC
2014/2015												
No split	0.80 ± 0.08	0.45 ± 0.04	0.80 ± 0.07	0.85 ± 0.02	0.46 ± 0.02	0.85 ± 0.01	0.61 ± 0.03	0.38 ± 0.03	0.61 ± 0.03	0.81 ± 0.03	0.43 ± 0.02	0.80 ± 0.03
Random	0.43 ± 0.05	0.21 ± 0.03	0.42 ± 0.06	0.71 ± 0.02	0.38 ± 0.02	0.71 ± 0.02	0.30 ± 0.09	0.19 ± 0.05	0.31 ± 0.08	0.43 ± 0.03	0.27 ± 0.03	0.42 ± 0.03
Temporal	0.03 ± 0.03	−0.08 ± 0.06	0.02 ± 0.03	−0.02 ± 0.02	−0.13 ± 0.03	−0.02 ± 0.02	−0.02 ± 0.04	−0.04 ± 0.03	−0.03 ± 0.04	−0.03 ± 0.06	−0.03 ± 0.05	−0.03 ± 0.05
Group-based	0.04 ± 0.03	−0.03 ± 0.06	0.03 ± 0.03	0.00 ± 0.02	−0.12 ± 0.04	−0.01 ± 0.02	0.00 ± 0.04	−0.02 ± 0.02	0.00 ± 0.04	−0.04 ± 0.03	−0.02 ± 0.03	−0.03 ± 0.02



(a) F1 score **(b) G_{mean}** **(c) MCC**

Figure 6: Performance Optimism (PO) for multiple data splitting techniques and ML/DL models.

results for the *no split* method confirm the expectations that a ML/DL model performs better on the training data than on held-out test data.

For the commonly used *random split* technique, the PO is consistently lower than for *no split*. However, it is still remarkably high. Most of all, the RF's F1/MCC and G_{mean} optimism amounts to 0.71 ± 0.02 and 0.38 ± 0.02, respectively. **For all tested models and classification metrics, the usage of the *random split* leads to a drastic overestimation of the predictive performance.**

In contrast, the *temporal split* shows considerably lower PO values (third row in Table 4). Within the standard deviation, all values are equal to or even smaller than 0. This is also true for the *group-based split* (last row in Table 4), except for one outlier, namely the F1 optimism of 0.04 ± 0.03 for the HGBC. However, the PO values for *temporal* and *group-based split* are consistently compatible within the limits of the standard deviation. **Consequently, for all tested models and classification metrics, the usage of the *temporal* or *group-based split* leads to a realistic estimation of the predictive performance.**

For all metrics, the PO is plotted in Figure 6, illustrating the much higher PO for *no split* and the *random split* compared to that of the *temporal* or *group-based* split, which is mostly equal to or even smaller than zero. Across all models, the G_{mean} optimism is lower compared to the other metrics. For the RF model and the *temporal* and *group-based* split, it even exhibits negative values of -0.13 ± 0.03 and -0.12 ± 0.04, respectively, indicating a clear underestimation of the model's predictive capabilities in terms of G_{mean}.

We conclude this section by relating these findings to the values of our presented leakage measure in Section 5.1.

- As expected, the baseline strategy *no split* consistently shows the highest PO, which is in line with the corresponding maximum leakage value $\tilde{\mathcal{L}} = 1.0$.
- The *random split* strategy shows to be susceptible to overestimating the model's performance. The temporal leakage and non-independence between train and test samples induced by this splitting strategy is a viable explanation, and is reflected by a value of $\tilde{\mathcal{L}} = 0.34$ for $\alpha = 0$ and $\tilde{\mathcal{L}} = 0.59$ for $\alpha = 1$, respectively.
- The remaining two split strategies *temporal* and *group-based split* showed to allow for a realistic estimation of the model performance. We are confident to conclude this from the fact that these strategies avoid temporal leakage by design. Thus, their test dataset better represents the distribution of the samples in the independent future dataset that we assume to be representative of the distribution of scientific interest. This aligns with the leakage value of $\tilde{\mathcal{L}} = 0.0$ for $\alpha = 0$.
- For $\alpha = 1$, the $\tilde{\mathcal{L}}$ value slightly differs between the *temporal* and *group-based split* strategy, which is not reflected in the PO evaluation results. This might be due to the fact that the independent future test dataset from 2016 corresponds to the task related to $\alpha = 0$, that is, predicting on data from already known groups or HDDs, as discussed in Section 5.1.

Overall, the extent of data leakage as indicated by our proposed measure $\tilde{\mathcal{L}}$ aligns with the experimental results, offering a viable explanation for the differences between the Performance Optimism (PO) induced by different splitting strategies.

6 Threats to Validity

In the following, we identify possible threats to the validity of the experimental results discussed in Section 5.

Comparing predictions for long-term future datasets carries the inherent risk of concept drift. However, since we assess model performance on a dataset from just one year later, where mostly the same HDDs are still in use, we are confident that any observed changes in model performance are unlikely to be impacted by concept drift.

The SMART feature *Life Left*, available only for SSDs, could serve as a proxy for predicting RUL. However, since our focus is solely on HDDs, and the objective of this paper is not to identify the best model, using only SSDs and incorporating this feature would most likely not alter the observed effect of PO.

To balance and reduce the size of the dataset, we down-sampled the *good* HDDs to 2.5%. This sampling approach may have introduced some bias, potentially impacting the results. However, random sampling was used, and the *good* class remained the largest class and, therefore, sufficiently represented. Moreover, the goal of this paper is not to find the best-performing model but to demonstrate the impact of data splitting on predictive power, data leakage, and PO. Thus, even if training bias were present and affected model performance, it would not influence the effect of performance optimism, as even a less accurate model can exhibit overconfidence.

Selecting the optimal hyperparameters for a model is essential. Although we aimed to define the hyperparameter search space as effectively as possible, there remains a risk that the search process was suboptimal. However, even if the best hyperparameters for each model were identified, these models would still be affected by PO although the models may have been slightly better in generalizing the data.

In this work, we focused specifically on temporal leakage and leakage induced by the non-independence between train and test samples. Thus, the proposed measure addresses only this type of leakage. Other forms of leakage are not captured by our measure. However, as outlined in previous sections, we mitigated various potential sources of leakage during preprocessing (for instance, fitting the scaler using only training data).

The assumptions of our proposed measure include the independence between measurements for different HDDs, even though, in general, it is possible that dependencies might exist, for instance, mutual interference due to excessive heat development of a HDD. However, we presume that these effects, if existing, are negligible.

Our proposed data leakage measure allows controlling the impact of training samples preceding test samples by parameter α. The intention is to make the measure adjustable to different tasks, that is, prediction on previously unknown, novel groups ($\alpha > 0$) vs. prediction on data from groups already encountered during training ($\alpha = 0$). We chose $\alpha \in \{0, 1\}$ to demonstrate the conceptual behavior for the two cases. However, selecting the right α value is not straightforward and requires further research.

The comparability of the leakage value $\tilde{\mathcal{L}}$ might be affected when comparing datasets with different sizes and sequence lengths. In addition, the actual amount of leakage might also depend on the underlying dependency structure of the data as well as other factors not captured by our measure since it is based on time and group information. Nevertheless, we are confident that for similar or equal dataset properties, like in the experiments conducted in this paper, the measure allows for a quantifiable assessment of data leakage.

7 Conclusion

Cloud computing's growing importance in recent years will continue in the future, especially as generative AI becomes more common. A key factor in the success of cloud computing is a reliable operation to which HDDs have a significant impact. Monitored data of HDDs can be used to predict failures and proactively replace affected elements. Failure prediction is an already well-researched field. However, clean preprocessing and the avoidance of data leakage cannot be verified for all publications. In this paper, we propose a novel measure for quantifying data leakage in prediction tasks. Further, we present four techniques for splitting a dataset into training and test data with different amounts of data leakage. We evaluate the impact of data leakage quantified with our proposed measure on the performance of a set of commonly used classification models. Our results consistently show that splitting techniques prone to data leakage induce an overestimation of predictive performance. They, therefore, suggest that the evaluation of deep learning experiments, in particular the choice of splitting technique, should be carried out with the utmost care and in alignment with the specific prediction task to avoid leakage. Overall, we were able to show the usefulness of the defined data leakage measure as well as its connection with different splitting techniques and performance optimism of prediction models.

In the future, we plan to assess the capabilities of our data leakage measure on use cases other than the HDD dataset, such as further predictive maintenance datasets. Currently, the definition of the measure's α value is not straightforward. Hence, we plan to conduct a comprehensive parameter study on a diverse set of use cases to showcase best practices in selecting the value. We also intend to refine our data leakage measure so that it also takes into account the length of the input sequence for the prediction. Further, we plan to extend the experiments to a more diverse set of splitting methods, for instance, variations of the random split with varying degrees of data leakage. Finally, we currently focus on data leakage within a group, for example, a specific HDD instance. We plan to investigate how data leakage can also be measured outside these group-type boundaries.

References

[1] Nicolas Aussel, Samuel Jaulin, Guillaume Gandon, Yohan Petetin, Eriza Fazli, and Sophie Chabridon. 2017. Predictive Models of Hard Drive Failures Based on Operational Data. In *16th IEEE International Conference on Machine Learning and Applications, ICMLA 2017, Cancun, Mexico, December 18-21, 2017*, Xue-wen Chen, Bo Luo, Feng Luo, Vasile Palade, and M. Arif Wani (Eds.). IEEE, 619–625. https://doi.org/10.1109/ICMLA.2017.00-92

[2] Backblaze. 2024. *Hard Drive Test Data.* Backblaze. Retrieved Oct 29, 2024 from https://www.backblaze.com/cloud-storage/resources/hard-drive-test-data

[3] James Bergstra, Daniel Yamins, and David D. Cox. 2013. Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures. In *Proceedings of the 30th International Conference on Machine Learning, ICML 2013, Atlanta, GA, USA, 16-21 June 2013 (JMLR Workshop and Conference Proceedings, Vol. 28)*. JMLR.org, 115–123. http://proceedings.mlr.press/v28/bergstra13.html

[4] Aniello De Santo, Antonio Galli, Michela Gravina, Vincenzo Moscato, and Giancarlo Sperli. 2022. Deep Learning for HDD Health Assessment: An Application Based on LSTM. *IEEE Trans. Comput.* 71, 1 (jan 2022), 69–80. https://doi.org/10.1109/TC.2020.3042053

[5] William Falcon and The PyTorch Lightning team. 2019. *PyTorch Lightning*. Lightning AI. https://doi.org/10.5281/zenodo.3828935
[6] Gartner. 2024. *Gartner Forecasts Worldwide Public Cloud End-User Spending to Surpass $675 Billion in 2024.* Gartner. Retrieved Oct 31, 2024 from https://www.gartner.com/en/newsroom/press-releases/2024-05-20-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-surpass-675-billion-in-2024
[7] Lihan Hu, Lixin Han, Zhenyuan Xu, Tianming Jiang, and Huijun Qi. 2020. A disk failure prediction method based on LSTM network due to its individual specificity. *Procedia Computer Science* 176 (2020), 791–799. https://doi.org/10.1016/j.procs.2020.09.074 Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 24th International Conference KES2020.
[8] Josu Ircio, Aizea Lojo, Jose A. Lozano, Usue Mori, and Jose A. Lozano. 2022. A Multivariate Time Series Streaming Classifier for Predicting Hard Drive Failures [Application Notes]. *IEEE Computational Intelligence Magazine* 17, 1 (2022), 102–114. https://doi.org/10.1109/MCI.2021.3129962
[9] Sayash Kapoor and Arvind Narayanan. 2023. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4, 9 (2023), 100804. https://doi.org/10.1016/j.patter.2023.100804
[10] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data* 6, 4, Article 15 (dec 2012), 21 pages. https://doi.org/10.1145/2382577.2382579
[11] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net. https://openreview.net/forum?id=Bkg6RiCqY7
[12] Sidi Lu, Bing Luo, Tirthak Patel, Yongtao Yao, Devesh Tiwari, and Weisong Shi. 2020. Making Disk Failure Predictions SMARTer!. In *18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*, Sam H. Noh and Brent Welch (Eds.). USENIX Association, 151–167. https://www.usenix.org/conference/fast20/presentation/lu
[13] Yingzhe Lyu, Heng Li, Mohammed Sayagh, Zhen Ming (Jack) Jiang, and Ahmed E. Hassan. 2021. An Empirical Study of the Impact of Data Splitting Decisions on the Performance of AIOps Solutions. *ACM Trans. Softw. Eng. Methodol.* 30, 4, Article 54 (jul 2021), 38 pages. https://doi.org/10.1145/3447876
[14] neptune.ai. 2024. *neptune.ai: experiment tracker*. neptune.ai. https://neptune.ai
[15] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 8024–8035. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html
[16] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* 12 (2011), 2825–2830. https://doi.org/10.5555/1953048.2078195
[17] Francisco Pereira, Daniel Teixeira, João Paulo Pordeus Gomes, and Javam C. Machado. 2019. Evaluating One-Class Classifiers for Fault Detection in Hard Disk Drives. In *8th Brazilian Conference on Intelligent Systems, BRACIS 2019, Salvador, Brazil, October 15-18, 2019*. IEEE, 586–591. https://doi.org/10.1109/BRACIS.2019.00108
[18] Sean Owen. 2021. *Use Hyperopt Optimally With Spark and MLflow to Build Your Best Model.* Databricks. Retrieved Aug 19, 2023 from https://www.databricks.com/blog/2021/04/15/how-not-to-tune-your-model-with-hyperopt.html
[19] Jing Shen, Jian Wan, Se-Jung Lim, and Lifeng Yu. 2018. Random-forest-based failure prediction for hard disk drives. *International Journal of Distributed Sensor Networks* 14, 11 (2018), 1550147718806480. https://doi.org/10.1177/1550147718806480 arXiv:https://doi.org/10.1177/1550147718806480
[20] Chuan-Jun Su and Yi Li. 2022. Recurrent neural network based real-time failure detection of storage devices. *Microsystem Technologies* 28, 2 (01 Feb 2022), 621–633. https://doi.org/10.1007/s00542-019-04454-8
[21] Guochao Wang, Yu Wang, and Xiaojie Sun. 2021. Multi-Instance Deep Learning Based on Attention Mechanism for Failure Prediction of Unlabeled Hard Disk Drives. *IEEE Transactions on Instrumentation and Measurement* 70 (2021), 1–9. https://doi.org/10.1109/TIM.2021.3068180
[22] Brian Wilson. 2018. *Cloud Storage Durability: What Do All Those 9s Mean?* Backblaze. Retrieved Sep 24, 2024 from https://www.backblaze.com/blog/cloud-storage-durability/
[23] Chang Xu, Gang Wang, Xiaoguang Liu, Dongdong Guo, and Tie-Yan Liu. 2016. Health Status Assessment and Failure Prediction for Hard Drives with Recurrent Neural Networks. *IEEE Trans. Comput.* 65, 11 (2016), 3502–3508. https://doi.org/10.1109/TC.2016.2538237
[24] Qibo Yang, Xiaodong Jia, Xiang Li, Jianshe Feng, Wenzhe Li, and Jay Lee. 2021. Evaluating Feature Selection and Anomaly Detection Methods of Hard Drive Failure Prediction. *IEEE Transactions on Reliability* 70, 2 (2021), 749–760. https://doi.org/10.1109/TR.2020.2995724

A Software Configuration

The most relevant Python packages used for the experiments are listed in Table 5. We used Neptune [14] for experiment tracking and monitoring. For further details, please refer to the well-documented project code in the following repository: https://github.com/DescartesResearch/Data-Leakage-HDD-Failure-Prediction

B Hyperparameter Tuning Results

This appendix section provides details about the hyperparameter ranges used for the hyperparameter search and the best hyperparameters found for the different ML and DL models and splitting strategies. These are listed in Tables 6 to 9.

Table 5: Adopted Python packages and versions.

Package	Version
lightning	2.2.4
lightning-utilities	0.11.2
neptune	1.10.4
numpy	1.26.4
pandas	2.2.2
ray	2.20.0
scikit-learn	1.4.2
scipy	1.13.0
torch	2.3.0
torchmetrics	1.3.2

Table 6: Best hyperparameters found per split type for HGBC.

Parameter Ranges	learning rate [0.01, 1.0)	max iter [50, 500)	max leaf nodes [5, 100)	max features [0.1, 1.0)	class weight {balanced, None}
No split	0.2196	321	92	0.8873	balanced
Random	0.2162	208	70	0.7093	balanced
Temporal	0.3217	162	98	0.8587	balanced
Group-based	0.4473	200	36	0.8500	balanced

Table 7: Best hyperparameters found per split type for RF.

Parameter Ranges	n estimators [50, 500)	max depth [5, 100)	max features [0.1, 1.0)
No split	437	92	0.1064
Random	237	77	0.1034
Temporal	375	48	0.1240
Group-based	55	21	0.1246

Table 8: Best hyperparameters found per split type for MLP.

Parameter Ranges	learning rate [1e-6, 1e-3)	hidden sizes $\{64, 96, ..., 256\}^3$
No split	0.000209	[224, 128, 224]
Random	0.000045	[192, 256, 192]
Temporal	0.000012	[96, 224, 64]
Group-based	0.000005	[160, 128, 192]

Table 9: Best hyperparameters found per split type for LSTM.

Parameter Ranges	learning rate [1e-6, 1e-3)	hidden size {64, 96, ..., 256}	num layers {1, 2}
No split	0.000109	256	2
Random	0.000279	128	2
Temporal	0.000218	192	1
Group-based	0.000005	224	1